

[9 Otázky a problémy pri použití REST](#)

[9.1 Služba z pohľadu klienta](#)

[9.2 Služba s komplexnou štruktúrou zdrojov](#)

[Príklad 5: Portál Svet kníh](#)

[Príklad 6: Portál Svet kníh - varianta s priamym prístupom ku knihám](#)

[9.3 Problém cyklických referencií](#)

[Príklad 7: Portál Svet kníh – varianta s obojstrannou asociáciou](#)

[Cvičenia](#)

9 Otázky a problémy pri použití REST

V prednáške na príkladoch ilustrujeme viaceré otázky, s ktorými sa môžeme stretnúť pri návrhu, implementácii a používaní REST služieb:

- *Ako môžu klienti zistiť URI zdrojov?*
- *Ako môže zistiť klient vnútornú štruktúru XML/JSON dát, ktoré akceptuje služba?*
- *Môže byť jeden zdroj prístupný prostredníctvom rôznych URI?*

9.1 Služba z pohľadu klienta

Pozrime sa na REST službu jedálne z pohľadu klienta, resp. vývojára klientskej aplikácie.

Ako môže klient zistiť URI zdrojov?

Aby klient mohol používať REST službu, musí poznať aspoň URI zdrojov, ktoré sprístupňuje. Informácie o zdrojoch a prístupových metódach sú po spustení služby vypublikované v tzv. [WADL](#) dokumente pod URI:

<ip-servera>/<aplikacia>/resources/application.wadl.

Vo WADL dokumente REST služby jedálne z predchádzajúcej prednášky nájdeme fragment s informáciami o koreňovom zdroji:

```
<resources base="http://localhost:8080/JedalenPost/resources/">
  <resource path="menu">
    <method id="getPocet" name="GET">
      <response>
        <representation mediaType="text/plain"/>
      </response>
    </method>
    <method id="pridajJedlo" name="POST">
      <request>
        <representation mediaType="text/plain"/>
      </request>
      <response>
        <representation mediaType="text/plain"/>
      </response>
    </method>
  </resource>
</resources>
```

Z fragmentu môžeme vyčítať, že relatívna URI koreňového zdroja je **menu** a pre narábanie s ním poskytuje REST služba HTTP metódy GET a POST, ktoré posielajú, resp. akceptujú textové dáta.

Z ďalšej časti WADL dokumentu môžeme zistiť, že koreňový zdroj menu je kolekcia a zdroje v tejto kolekcii majú relatívnu URI, ktorá je číslo (pozri XML-element param):

```
<resource path="{cislo}">
  <param name="cislo" style="template" type="xs:int"/>
  <method id="odstranJedlo" name="DELETE"/>
  <method id="getJedlo" name="GET">
    <response>
      <representation mediaType="text/plain"/>
    </response>
  </method>
</resource>
```

```
</resource>
```

Ako klient zistí URI zdrojov v kolekcii?

Klient môže poslať požiadavku GET pre koreňový zdroj `menu` a v odpovedi dostane informáciu o aktuálnom počte jedál (pozri špecifikáciu [služba jedálne - varianta POST](#)). Z toho by mohol usúdiť, že relatívne URI jedál by mohli byť ich poradové čísla. Stále však nie je jasné, či sa číslovanie začína od 0 alebo od 1. Overiť sa to dá len ďalším experimentovaním.

Ešte zložitejšie to je však v prípade [služba jedálne – varianta POST s ukladáním v databáze](#), kde je relatívna URI jedla číselný kľúč generovaný databázou. Číselné kľúče tu nemusia ísť za sebou a tak klient len ťažko uhádne URI jedál v ponuke. Riešením by bolo, keby GET metóda pre `menu` vrátila zoznam kľúčov jedál v `menu` (teda relatívnych URI zdrojov v kolekcii). Pozri cvičenie 8.6.

Ako klient zistí vnútornú štruktúru XML/JSON dát?

WADL súbor poskytuje klientom popri URI zdrojov aj formát (MIME-type) reprezentácie ich stavu pre jednotlivé prístupové metódy. V prípade štruktúrovaných dát vo formátoch JSON alebo XML však neposkytuje žiadnu ďalšiu informáciu o ich vnútornej štruktúre. Ak nemáme k dispozícii špecifikáciu dátových tried na strane servera, nevieme celkom dobre vytvoriť dátovú triedu, do ktorej by sme mohli dáta posielané vo formáte XML alebo JSON načítať. Môžeme ich načítať ako textový reťazec. Pomocou volania GET tak získať nejaké príklady dát a podľa nich skúsiť navrhnúť vhodnú dátovú triedu.

Ilustrujeme si tento postup pri návrhu a implementácii klienta služby jedálne – varianta PUT s ukladáním v databáze. (Implementácia služby bola úlohou na cvičenia 8.8.)

Predpokladajme, že služba je spustená a v databáze sa už nachádzajú dve jedlá:

- Guláš s cenou 6.5 bez alergénov a opisu
- Špagety s cenou 5.5 a alergénom 7, bez opisu

GET metóda pre kolekciu `menu` nám vráti JSON obsahujúci názvy jedál v ponuke:

```
["Gulas", "Spagety"]
```

Volanie GET pre podzdroj `menu/Gulas` vráti informácie o tomto jedle vo formáte XML:

```
<jedlo>
  <nazov>Gulas</nazov>
  <cena>6.5</cena>
</jedlo>
```

podobne pre `menu/Spagety`

```
<jedlo>
  <nazov>Spagety</nazov>
  <cena>5.5</cena>
  <alergen>7</alergen>
</jedlo>
```

Na základe týchto experimentov by sme pre implementáciu klienta mohli navrhnúť triedu `Jedlo` s tromi dátovými členmi.

```
class Jedlo implements Serializable {
    private String nazov;
    private double cena;
    private int alergen;
```

V skutočnosti však informácia o jedle môže mať aj ďalšiu položku – `popis`. Navyše, alergénov tam môže byť aj viac. Správne navrhnutá trieda by mala vyzeráť tak, ako na strane servera.

```
class Jedlo implements Serializable {
    private String nazov;
    private String popis;
    private double cena;
    private List<Short> alergen;
```

Detailné informácie o štruktúre XML reprezentácie zdroja však WADL dokument neobsahuje. Pokiaľ nemáme k dispozícii inú podrobnú dokumentáciu k službe, môžeme navrhnúť triedu len na základe experimentov, pozri cvičenie 8.7.

9.2 Služba s komplexnou štruktúrou zdrojov

Ďalšie problémy a otázky si ilustrujeme na rozsiahlejšom informačnom systéme založenom na REST poskytujúcom viacúrovňovú hierarchiu zdrojov.

Príklad 5: Portál Svet kníh

Portál Svet kníh je REST servis, ktorý poskytuje informácie o kníhkupectvách a knihách v ponuke.

Špecifikácia služby:

Informácie o kníhkupectve:

- názov - je jeho jednoznačná identifikácia
- ponuka - obsahuje zoznam ponúkaných kníh
- adresa

Informácie o knihe:

- isbn - jednoznačný identifikátor publikácie
- titul
- cena

Koreňový zdroj portálu reprezentuje kolekciu kníhkupectiev a má URI [SvetKnih](#). Kníhkupectvá predstavujú samostatné zdroje s URI [SvetKnih/{obchod}](#), kde [obchod](#) je názov kníhkupectva. Informácie o knihách v ponuke kníhkupectva predstavujú zdroje s URI [SvetKnih/{obchod}/{isbn}](#), kde [isbn](#) je isbn knihy.

Pre prístup k zdrojom poskytuje služba nasledujúce metódy:

Pre [SvetKnih](#)

[GET](#) vráti zoznam názvov obchodov (MIME application/json)

Poznámka. Zoznam reťazcov nie je možné vrátiť vo formáte XML.

Pre [SvetKnih/{obchod}](#)

[GET](#) vráti všetky informácie o obchode a ponúkaných knihách (MIME application/xml)

Pre [SvetKnih/{obchod}/{isbn}](#)

[GET](#) vráti všetky informácie o knihe (MIME application/xml)

[PUT](#) - umožní modifikovať informácie o knihe.

Implementácia

Dátové triedy by mohli vyzerat' nasledovne:

```
public class Kniha {  
    private String isbn;  
    private String titul;  
    private double cena;  
}
```

```
public class Knihkupectvo {  
    private String nazov;  
    private String adresa;  
    @XmlElementWrapper(name="ponuka")  
    @XmlElement(name="kniha")  
    private List<Kniha> ponuka;  
}
```

V prvej verzii budeme informácie uchovávať zatiaľ len v pamäti v zozname obchodov, ktorý hneď inicializujeme dátami pre testovanie:

```
obchody = new ArrayList<>();  
Kniha k1 = new Kniha("111", "Hobit", 6.5);  
Kniha k2 = new Kniha("222", "Pipi dlha pancucha", 7.5);  
Kniha k3 = new Kniha("333", "Mach a Sebestova", 7.5);  
Knihkupectvo m = new Knihkupectvo("Martinus");  
Knihkupectvo p = new Knihkupectvo("PantaRhei");  
m.getPonuka().add(k1);
```

```

m.getPonuka().add(k2);
p.getPonuka().add(k1);
p.getPonuka().add(k3);
obchody.add(p);
obchody.add(m);

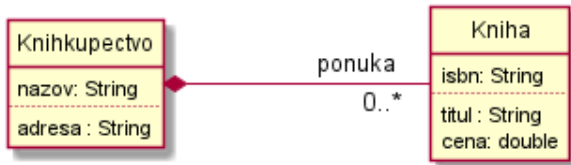
```

Implementácia metódy PUT a modifikácia údajov o knihách

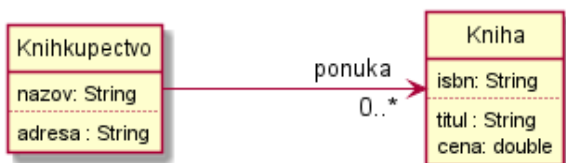
Pri implementácii metódy PUT si však musíme položiť jednu dôležitú otázku:

Má zmena ceny knihy v jednom kníhkupectve spôsobiť zmenu aj vo všetkých ostatných?

Inými slovami: Je vzťah medzi kníhkupectvom a knihami v ponuke **kompozícia**



alebo obyčajná **asociácia**.

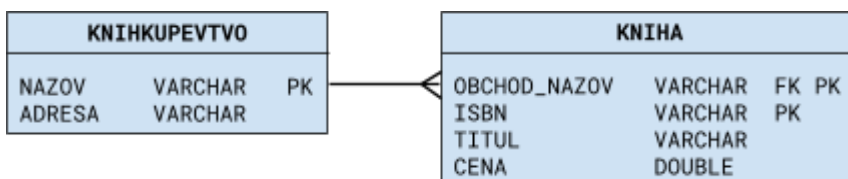


Poznámka: Od odpovede na túto otázku bude závisieť aj spôsob implementácie metód pre vkladanie a odstraňovanie kníh. (Pozri cvičenia)

Varianta 1 - Kompozícia:

V tomto prípade má každé kníhkupectvo vo svojej ponuke vlastné kópie kníh. Metódu PUT môžeme implementovať jednoducho tak, že pôvodnú knihu z ponuky kníhkupectva odstránime a novú inštanciu knihy vložíme do ponuky. Modifikácie údajov o knihe sa tak týkajú a prejavajú len v ponuke daného kníhkupectva.

Pre perzistentné uloženie informácií v databáze by mala mať kniha kľúč zložený z ISBN knihy a názvu kníhkupectva. Tabuľka KNIHA totiž v tomto prípade nepredstavuje samostatnú entitu.



Varianta 2 - Asociácia:

V tomto prípade existuje pre každú knihu v pamäti len jedna inštancia. Na túto inštanciu odkazujú všetky kníhkupectvá, ktoré ju majú v ponuke.

Ak sú informácie udržiavané len v pamäti, je implementácia metódy PUT je zložitejšia: Pôvodnú referenciu na knihu neodstraňujeme, len v nej aktualizujeme zmenené dátové členy. Takto sa zmeny prejavajú aj v ponuke všetkých kníhkupectiev, ktoré na ňu odkazujú.

Druhá možnosť je referenciu na knihu odstrániť a nahradiť odkazom na novú inštanciu. Treba to však urobiť pre všetky kníhkupectvá, ktoré ju ponúkajú.

Pri perzistentnom uložení informácií v databáze sú KNIHA aj KNIHUKUPECTVO dve plnohodnotné entity, medzi ktorými je vzťah N:N realizovaný prepojujovou tabuľkou:



Implementovať **PUT** môžeme teraz jednoducho pomocou metódy **merge** entity managera.

Implementácia varianty **Kompozícia** s uložením údajov len v pamäti : Projekt [SvetKnihV1pojo](#).

Implementácia varianty **Asociácia** s uložením údajov v databáze : Projekt [SvetKnihV2db](#).

Varianty z pohľadu klienta

Keď klient zavolá metódu PUT pre URI [SvetKnih/{obchod}/{isbn}](#) zrejme očakáva, že zmení cenu len knihy len v danom obchode. V prípade implementácie ako asociácie, bude užívateľ asi prekvapený, že sa pritom cena zmení vo všetkých obchodoch. (*O implementačných detailoch sa z WADL súboru nič nedozvie.*)

Ak chceme, aby portál umožňoval meniť cenu knihy naraz vo všetkých obchodoch, nie je vhodné implementovať metódu PUT pre URI ponuky obchodu [SvetKnih/{obchod}/{isbn}](#).

Vhodnejší by bol priamy prístup napr. cez URI [SvetKnih/kniha/{isbn}](#)

Príklad 6: Portál Svet kníh - varianta s priamym prístupom ku knihám

Táto varianta služby realizuje vzťah medzi kníhkupectvom a knihami ako asociáciu medzi samostatnými zdrojmi, čo umožňuje pristupovať ku knihám nielen prostredníctvom kníhkupectva, ale aj priamo.

Implementuje nasledujúce metódy

Pre URI [SvetKnih](#)

GET vráti zoznam názvov obchodov vo formáte JSON.

Pre [SvetKnih/{obchod}](#)

GET vráti všetky informácie o obchode a ponúkaných knihách vo formáte XML.

Pre [SvetKnih/{obchod}/{isbn}](#)

GET vráti všetky informácie o knihe vo formáte XML.

Pre [SvetKnih/kniha/{isbn}](#)

PUT modifikuje informácie o knihe (vo všetkých kníhkupectvách). Akceptuje informácie o knihe vo formáte XML.

Do implementácie služby by sme mohli doplniť tiež GET metódy pre priame vyhľadávanie kníh:

Pre [SvetKnih/kniha](#)

- GET** dostane podreťazec ako parameter požiadavky a vráti zoznam isbn všetkých kníh, ktoré majú podreťazec v názve. Formát JSON.

Pre [SvetKnih/kniha/{isbn}](#)

- GET** vráti všetky informácie o knihe vo formáte XML.

Poznámka: Prístup ku knihám prostredníctvom URI obchodu [SvetKnih/{obchod}/{isbn}](#) teda nebude vôbec potrebný.

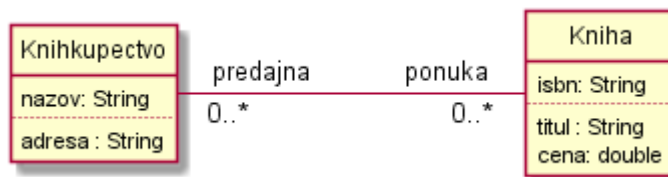
Implementáciu príkladu 6 prenechávame na čitateľa, pozri cvičenie 9.2.

9.3 Problém cyklických referencií

Príklad 7: Portál Svet kníh – varianta s obojstrannou asociáciou

Pripomeňme, že asociácia medzi triedami `Kníhkupectvo` a `Kniha` v príklade 6 je jednosmerná (pozri UML diagram hore). Kníhkupectvo má zoznam ponúkaných kníh, ale kniha nemá žiadny dátový člen, ktorý by odkazoval na kníhkupectvá, kde je v ponuke. Klient, ktorý si informácie o knihe načíta priamo prostredníctvom URI [SvetKnih/kniha/{isbn}](#), teda nevie priamo zistiť, v ktorých kníhkupectvách je v ponuke.

Ak by v objektovom modeli bola asociácia medzi kníhkupectvom a knihami obojsmerná, informáciu o predajniach, ktoré ju ponúkajú, by poskytovala priamo kniha. To môžeme dosiahnuť jednoducho doplnením dátovej triedy `Kniha` o zoznam kníhkupectiev.



Zmeny v implementácii prístupových metód, zdá sa, nie sú nutné.

Keď však takto upravenú službu spustíme a pomocou požiadavky GET pre URI [SvetKnih/{obchod}](#) sa pokúsime zistiť informácie o niektorom kníhkupectve, vráti služba kritickú výnimku: `HTTP Status 500 - Internal Server Error`

Problém je v tom, že obojstranná asociácia vytvára cyklus referencií, ktorý by pri serializácii do XML alebo JSON vytvoril stromovú štruktúru nekonečnej hĺbky.

Poznámka: Bohužiaľ, chybová správa klienta integrovaného v prostredí NetBeans neuvádza, v čom je problém. Môžeme ho však reprodukovat' jednoduchou aplikáciou, ktorá pomocou komponentu JAXB zapisuje dátové objekty do XML. Ak má kníhkupectvo v ponuke nejakú knihu, jeho zápis do XML zlyhá s chybovou správou: `A cycle is detected in the object graph`.

Riešenie

Potrebné je zamedziť serializáciu referencovaných objektov z jednej strany asociácie pomocou anotácie `@XmlTransient`:

```
@XmlTransient
@ManyToMany
private List<Kniha> ponuka;
```

Teraz však reprezentácia kníhkupectva, ktorú vracia metóda GET pre URI [SvetKnih/{obchod}](#) neobsahuje ponuku kníh.

Poskytnúť zoznam kníh, ktoré ponúka dané kníhkupectvo, by mohla ďalšia GET metóda pre špeciálne URI, napr. [SvetKnih/{obchod}/ponuka](#). Stačí, ak vráti zoznam ISBN kníh v ponuke. K detailným informáciám o konkrétnej knihe má klient prístup prostredníctvom URI [SvetKnih/kniha/{isbn}](#).

Riešenie pomocou virtuálnej dátovej položky

Virtuálna dátová položka (virtual property) je dátová položka, ktorej stav je prístupný len prostredníctvom prístupových metód (getter a setter) rovnako ako štandardné dátové položky java-bean triedy. Na rozdiel od nich, jej stav však nie je uložený vo vlastnom dátovom člene, ale prístupová get-metóda si ho vypočíta pri každom zavolaní. Odvodzuje si ho zo stavu ostatných dátových členov.

V triede `Knihkupectvo` môžeme implementovať get-metódou pre virtuálnu položku poskytujúcu zoznam ISBN kníh v ponuke takto:

```
@XmlElementWrapper(name = "ponuka")
@XmlElement(name = "isbn")
public List<String> getIsbnList() {
    List<String> isbnList = new ArrayList<>();
    for(Kniha k: ponuka) {
        isbnList.add(k.getIsbn());
    }
    return isbnList;
}
```

Aby sa do XML serializovali aj virtuálne položky, je potrebné použiť anotáciu triedy.

`@XmlAccessorType(XmlAccessType.PROPERTY)`:

```
@Entity
@XmlRootElement
@XmlAccessorType(XmlAccessType.PROPERTY)
public class Knihkupectvo implements Serializable {
```

Poznámka: Pri použití voľby `XmlAccessType.PROPERTY` treba presunúť všetky JAXB anotácie z dátových členov na getter metódy.

Implementáciu **príkladu 7** s využitím virtuálnej dátovej položky nájdete v projekte [SvetKnihV3](#)

Cvičenia

Cvičenie 9.1 Implementácia príkladu 5 s ukladaním dát v databáze

Reimplementujte [Príklad 5: Portál Svet kníh](#) varianta 1 – kompozícia tak, aby, informácie o knihách a kníhkupectvách budú uložené persistentne v relačnej databáze.

Cvičenie 9.2 Implementácia príkladu 6

Implementujte portál podľa špecifikácie [Príklad 6 - varianta s priamym prístupom ku knihám](#), pričom informácie o knihách a kníhkupectvách budú

- uchovávané len v pamäti servera,
- uložené persistentne v relačnej databáze.

Cvičenie 9.3 Odstránenie knihy z ponuky kníhkupectva

Požiadavkou DELETE pre URI `SvetKnih/{obchod}/{isbn}` by klient mohol použiť na odstránenie knihy z ponuky kníhkupectva. Doplňte túto metódu do implementácií služby z príkladov 5 a 6 (resp. cvičení 9.1 a 9.2).

Cvičenie 9.4 Pridanie existujúcej knihy ponuky kníhkupectva

Vo variante služby s priamym prístupom ku knihám ([Príklad 6](#)) môže klient použiť metódu PUT aj na vytvorenie novej knihy v portáli kníh. Nová kniha však ešte nebude v ponuke žiadneho kníhkupectva.

Navrhnite a implementujte ďalšie rozšírenie služby tak, aby poskytovala metódu aj na zaradenie existujúcej knihy do ponuky kníhkupectva. Implementujte tiež metódy na odstránenie knihy z ponuky kníhkupectva, ako aj na úplné odstránenie knihy z portálu.

Navrhnite tiež spôsob, ktorý by aj vo variantoch služby z [Príkladu 5](#) umožňoval klientom pridať do ponuky kníhkupectva knihu, ktorá existuje už v ponuke iného kníhkupectva.